

Week 7: Conjoint Questions

October 7, 2026

THE POINT OF THIS WEEK

So far your survey has been markdown and simple questions. This week it gets a *design*: a table of attribute combinations generated in R, fed into *surveydown* so each respondent sees different, systematically-varied choice questions. Two steps, always in this order:

1. **Generate the design in R** with `{cbcTools}` — a data frame, one row per alternative shown.
2. **Display it in *surveydown*** — read that data frame, filter it to the current respondent, and render each choice question.

The choice-question **design** (who sees what) and the choice-question **display** (how it looks on screen) are two separate steps using two separate tools. Get the design right in R first — the display just reads it.

STEP 1: BUILDING THE DESIGN (CBCTOOLS)

Profiles → design

```
library(cbcTools)

profiles <- cbc_profiles(
  price = c(15, 20, 25),
  fuelEconomy = c(20, 25, 30),
  powertrain = c("Gasoline", "Electric")
)

design <- cbc_design(
  profiles = profiles,
  n_resp = 500, # number of respondents
  n_alts = 3,   # alternatives per question
  n_q = 8       # questions per respondent
)

write_csv(design, file.path("data",
  "choice_questions.csv"))
```

`cbc_profiles()` is the full factorial of attribute levels. `cbc_design()` samples question-level combinations from it — one row per respondent/question/alternative.

Three design variants

Variant	What it means	How
Basic	Any attribute combo can appear in any option	default <code>cbc_design()</code>
Labeled	One attribute is fixed per option position (e.g. option 1 is always Gasoline)	<code>label = "powertrain"</code>
No-choice	Respondent can opt for “none of these”	<code>no_choice = TRUE</code>

Attribute-specific features & restrictions

Some levels only make sense combined with certain other levels — e.g. *driving range* only applies to electric vehicles, not gasoline ones. Add restrictions with `cbc_restrict()` on the `profiles` before designing.

Restrictions can create **imbalanced** designs (some levels appear more often than others). Pass `balance_by` to `cbc_design()` to correct for this.

THE DESIGN DATA FRAME'S SHAPE

Each row is one **alternative** within one **question** within one **respondent**:

Column	Meaning
<code>respID</code>	Respondent number
<code>qID</code>	Question number (within respondent)
<code>altID</code>	Which option this row is (1, 2, 3...)
attribute columns	One column per attribute (<code>price</code> , <code>type</code> , ...)

A `no_choice` row uses a sentinel `altID` (e.g. 4) with blank attributes.

STEP 2: DISPLAYING QUESTIONS (SURVEYDOWN)

Six steps, every time:

1. **Read the design file:**
`readr::read_csv("choice_questions.csv")`
2. **Sample & store a respondent ID:** `sample(design$respID, 1)`, then `sd_store_value(respondentID, "respID")` so it joins back to *design* later.
3. **Filter the design** to that one `respID`.
4. **Build an options function** — takes a filtered data frame, returns a named vector: each **name** is the HTML/markdown shown, each **value** is the stored answer code (`"option_1", ...`).
5. **Call it once per question** (`qID == 1`, `qID == 2`, ...) to make `cbc1_options`, `cbc2_options`, etc.
6. **Pass each to `sd_question()`**, one per question page.

Two display layouts

Layout	How options render	Question code
Buttons	Each option is its own clickable card (image + attributes via <code>glue()</code>)	<code>type = 'mc_buttons'</code> , labels built with <code>glue()</code>
Table	All options shown side-by-side in one <code>kable()/kbl()</code> table; buttons below just pick 1/2/3	<code>sd_output()</code> renders the table, then a separate <code>mc_buttons</code> question with plain "Option 1"/"Option 2" labels

```
sd_question(
  type = 'mc_buttons', id = 'cbc_q1',
  label = "(1 of 6) Which would you choose?",
  option = cbc1_options
)
```

Worked example: the options function (buttons layout)

```
make_cbc_options <- function(df) {
  alt1 <- df |> filter(altID == 1)
  alt2 <- df |> filter(altID == 2)
  alt3 <- df |> filter(altID == 3)
  options <- c("option_1", "option_2", "option_3")
  names(options) <- c(
    glue("**Opt 1** <img src='{alt1$image}' width=100>{alt1$type}, $ {alt1$price}/lb"),
    glue("**Opt 2** <img src='{alt2$image}' width=100>{alt2$type}, $ {alt2$price}/lb"),
    glue("**Opt 3** <img src='{alt3$image}' width=100>{alt3$type}, $ {alt3$price}/lb")
  )
  return(options)
}
```

`{glue}` interpolates `{alt1$image}` etc. straight into the HTML string — one alternative's attribute values become one option's displayed card.

Quarto tips from today

`kable()` turns a data frame into a markdown table; `t(df)` transposes a small one so attributes run down rows instead of across wide columns. `{kableExtra}`'s `kbl()` %>% `kable_styling()` adds striping/hover on top. Footnotes: `text^[footnote content]` is the simple, class-approved approach; bibtex citations (`[@citekey]`) are the more complete option.

YOUR TURN TODAY

1. Open the **conjoint_buttons** or **conjoint_tables** template folder in Positron.
2. With your team, decide: regular or labeled design? Include a “none” option? Any restrictions?
3. Edit `code/1_make_choice_questions.R` to build your own design, then wire the result into `survey.qmd` using whichever layout (buttons or table) fits your attributes.

ERRORS YOU MAY HIT THIS WEEK

Message	Usual cause
Blank/broken images	Filename doesn't match <code>images/</code> , or path not pasted on
Wrong option for wrong respondent	Forgot to filter <code>design</code> by sampled <code>respID</code> first
could not find function " <code>cbc_design</code> "	<code>{cbcTools}</code> not installed
Table shows literal <code></code> text	Missing <code>escape = FALSE</code> in <code>kbl()</code>

BEFORE NEXT CLASS

- Finish this week's choice-question design and get it rendering in your team's `surveydown` survey (**Survey Plan 2** milestone).
- Do [HW 6](#), due Monday **October 12**.
- **Pilot Survey** project deliverable due **October 13**.

You should be able to:

- Explain the difference between generating a design (`cbcTools`) and displaying it (`surveydown`).
- Build a basic, labeled, or no-choice conjoint design with `cbc_design()`.
- Filter a design to one respondent and turn it into `sd_question()` options, in either the buttons or table layout.